

Resolução dos Exercícios

Capítulo: Testes e condições

1. Por que razão variáveis do tipo *float* não devem armazenar valores lógicos

Devido à imprecisão dos `floats`, o uso desse tipo para valores lógicos é desaconselhável, pois os erros de arredondamento podem gerar valores distintos dos esperados inicialmente para verificar condições.

2. Indique quais das seguintes afirmações são verdadeiras e quais são falsas.

- 2.1 O `else` de um `if` é facultativo. - **verdadeira**
- 2.2 Num `if` são necessários parênteses em torno da condição. - **verdadeira**
- 2.3 O `if` pode conter a palavra `then` opcionalmente. - **falsa**
- 2.4 Tanto a componente `if` como a componente `else` só podem conter uma única instrução. - **verdadeira** (Para ser executada mais de uma instrução é necessário formar um bloco `{ }`, senão somente uma instrução estará atrelada as componentes).
- 2.5 O `if` tem que estar numa linha diferente do `else` - **falsa**
- 2.6 Na condição do `if` pode ser colocada uma constante, uma variável ou uma expressão. - **verdadeira**

3. Como consegue uma instrução *if-else* saber onde termina o *if* e começa o *else*, ou se o *if* tem ou não um *else*.

Se não houver delimitação por `{ }`, cada componente do *if-else* terá apenas uma instrução associada a si. Havendo `{ }`, todas as instruções presentes no bloco estarão associadas à estrutura em questão. Assim, o compilador identifica o escopo de cada bloco. Além disso, cada `else` é associado ao

último `if` declarado no mesmo escopo, o que permite determinar se um `if` possui ou não um `else` correspondente.

4. Um bloco pode ser constituído por apenas uma instrução?

Sim. Um bloco é definido pela delimitação de um par de `{}`. Embora estruturas em C, como o `if`, dispensem o uso de `{}` quando possuem apenas uma instrução, nada impede que um comando único seja envolto por elas para formar um bloco.

5. Depois de um bloco é obrigatório o uso de `;` ?

Não. O uso de `;` é obrigatório apenas para as instruções que compõem o bloco. O fechamento da chave `}` já sinaliza ao compilador o fim da estrutura, dispensando o ponto e vírgula após ela.

6. Exercício de Exame

Existe alguma diferença no funcionamento dos seguintes trechos?

```
if (x==0)           if (x=0)
    printf("X");      printf("X");
else                else
    printf("Y");      printf("Y");
```

Sim, existe uma diferença fundamental. No primeiro trecho, a condição testada é se `x` é igual a 0 (`==`). Se for verdadeiro, a instrução do `if` é executada; caso contrário, executa-se o `else`.

No segundo trecho, a variável `x` está em um contexto de atribuição (`=`). O valor 0 é atribuído a `x`, e o resultado dessa expressão é o próprio valor atribuído (0). Como em C o valor 0 representa falso, a condição do segundo `if` será sempre avaliada como falsa.

Assim, no primeiro código o resultado depende do valor prévio de `x`. No segundo, independentemente do valor inicial de `x`, ele será zerado e a instrução do `else` será sempre a executada.

7. Exercício de Exame

A indentação facilita o processo de

- ☐ a) Compilação
- ☐ b) Linkagem
- ☐ c) Execução
- ☒ d) Programação

8. Um programa indentado é, em geral:

- ☐ a) Mais rápido de executar que outro que não o seja.
- ☐ b) Mais lento de executar que outro que não o seja.
- ☒ c) Mais legível que outro que não seja indentado.
- ☐ d) Menos legível que outro que não seja indentado.

9. Exercício de Exame

Sempre que um compilador detecta um código mal indentado:

- ☐ a) Emite um erro.
- ☐ b) Emite um "WARNING".
- ☐ c) Escreve um arquivo corretamente indentado.
- ☒ d) Um compilador não faz qualquer tipo de verificação da indentação.

10. Indique duas vantagens e duas desvantagens do `if-else` em relação ao `switch`

VANTAGENS:

`if-else` permite testar expressões complexas com operadores lógicos e relacionais, enquanto o `switch` trabalha apenas com valores constantes. Também é possível avaliar múltiplas condições simultâneas em uma estrutura `if-else`.

DESVANTAGENS:

Quando há muitas condições para testes, o aninhamento de `if-else` torna o código difícil de ler, sendo o `switch` uma alternativa mais limpa e organizada. É comum haver confusão ao identificar qual `if` está associado a qual `else`

quando há muitos `if` e `else` no código. Com o uso do `switch`, este problema é evitado.

11. Será que a instrução `break` quando apresentada dentro de um `if`, passa a execução automaticamente para o `else`?

Não. A instrução `break` deve estar associada a uma estrutura de repetição (`for`, `while`, `do-while`) ou a um `switch`, caso contrário, o compilador acusará um erro. Mesmo que o `if` esteja dentro de uma estrutura de repetição, o `break` manterá seu comportamento padrão: ele interromperá a execução do laço de repetição por completo, e não desviará o fluxo para o `else`.

12. Qual o valor lógico que as seguintes expressões enviam para o `if`?

- a) `if (10 == 5)` - falso
- b) `if ((2+3) == -( -2 -3))` - verdadeiro
- c) `if (x = 5)` - verdadeiro
- d) `if (x = 0)` - falso

13. Supondo `x=4`, `y=6` e `z=-1`, qual o valor lógico das seguintes expressões:

- a) `if (x == 5)` - falso
- b) `if (x == 5 || z < 0)` - verdadeiro
- c) `if (y - x + z -1)` - falso
- d) `if (x == 4 || y >= z && ! (z))` - verdadeiro

14. Escreva, utilizando um único `if`, o seguinte código.

```
if (x == 0)
    if (y <= 32)
        printf("Sucesso!!!");
```

```
if (x == 0 && y <= 32)
    printf("Sucesso!!!");
```

15. Identifique os erros de compilação que seriam detectados nos seguintes programas:

15.1:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include <stdio.h>
main()
{ int x;
  if (x == 0)
      break;

  else
      printf("X não é zero \n");
}
```

O erro de compilação ocorre porque a instrução `break` está sendo utilizada fora de um contexto válido. Em C, o `break` deve, obrigatoriamente, estar contido dentro de uma estrutura de repetição ou de um comando `switch`.

15.2:

```
/*
```

```

* Copyright: Asneira Suprema Software!!!
*/

#include <stdio.h>
main()
{
    int x;
    if (x==0) then
        printf("X é zero\n");
    else
        printf("X não é zero\n");
}

```

A palavra `then` não é uma instrução válida na linguagem C. O compilador acusará um erro de sintaxe, pois a estrutura correta do `if` não utiliza esse termo.

15.3:

```

/*
* Copyright: Asneira Suprema Software!!!
*/

#include <stdio.h>
main()
{
    int x;
    switch (x)
    {
        case 1: printf("um"); break;
        case 2: printf("um"); break;
        else: printf("Nem um nem dois");
    }
}

```

O comando `else` deve estar associado a um `if`, não faz parte do comando `switch`, por isso o programa terá um erro e não compilará.

16. Escreva um programa, de quatro formas distintas, que leia um inteiro e indique se esse inteiro é ou não igual a zero

[Ver código-fonte \(exe09.c\)](#)

17. Reescreva o programa anterior com um *switch*.

[Ver código-fonte \(exe09.c\)](#)

18. Escreva um programa que verifique se um ano é bissexto ou não.

[Ver código-fonte \(exe09.c\)](#)

19. Escreva um programa que indique o número de dias existentes em um mês (fevereiro = 28 dias).

- 19.1 Usando apenas a instrução de teste *if-else*

[Ver código-fonte \(exe09.c\)](#)

- 19.2 Usando o *switch*

[Ver código-fonte \(exe09.c\)](#)

- 19.3 Usando o *switch* sem qualquer *break*

[Ver código-fonte \(exe09.c\)](#)

20. Escreva um programa que leia uma data e verifique se esta é válida ou não.

[Ver código-fonte \(exe10.c\)](#)